

# CodeQuizz Requisitos de Software

## Arquitetura regras e critérios de aceitação

Projeto de Ronald Araújo Corrêa. Este documento traduz o GDD do CodeQuizz em comportamento verificável do software. Ele define telas, dados, regras de jogo, sorteio de questões, persistência e publicação. A versão inicial executa no navegador, sem backend próprio.

A solução definida para o MVP é front end com HTML, CSS, JavaScript em módulos, JSON para conteúdo, SVG para mapas e localStorage para perfis e progresso. Git e GitHub registram as versões, e GitHub Pages publica os arquivos estáticos. Node.js, PHP e MySQL não são necessários para executar o jogo publicado.

## Prioridades e status

M significa requisito obrigatório no MVP. E significa expansão documentada para uma versão posterior. Os requisitos são especificações planejadas; não indicam implementação concluída. Os valores numéricos são configuração inicial para testes e devem permanecer sincronizados com o GDD.

| Parte   | Conteúdo                                          |
|---------|---------------------------------------------------|
| 1 a 2   | Escopo, arquitetura e módulos                     |
| 3 a 6   | Requisitos funcionais                             |
| 7 a 8   | Regras de negócio e seleção dinâmica              |
| 9 a 11  | Modelo de conteúdo, progresso e exemplos JSON     |
| 12 a 13 | Persistência, estados e requisitos não funcionais |
| 14 a 16 | Aceitação, publicação, evolução e referências     |

Versão documental 1.0. Data de referência: 01 de outubro de 2026. Os identificadores RF01 a RF26 preservam a sequência da discussão inicial, com escopo ajustado para mundos e fases.

# 1 Escopo e limites do MVP

## Funcionalidades incluídas

- Tela inicial, ajuda, perfis locais, apelido e dois avatares padrão.
- Central, seleção de mundos, mapa SVG e lista acessível de fases.
- CAMP60 com oito fases; quatro regiões futuras apresentadas como Em desenvolvimento.
- Partidas com alternativas, feedback, explicação, sorteio por dificuldade e histórico de questões.
- Dica, pulo, pontos, XP, níveis, CodeCoins, sequência, estrelas e desbloqueios.
- Objetivos permanentes, Sala de Troféus, estatísticas e ranking de perfis locais.
- Continuação de partida, salvamento, exportação e importação de progresso.
- Interface responsiva, controles de som e animação e publicação no GitHub Pages.

## Limites explícitos

O perfil local não é uma conta de servidor. Não são solicitados email, senha ou dados confidenciais. Ranking local reúne perfis do mesmo navegador e origem. Não há sincronização automática entre aparelhos, administração remota, banco centralizado ou autenticação real.

Perguntas e respostas fazem parte dos arquivos entregues ao navegador. O MVP é uma ferramenta de aprendizagem; seus dados locais não constituem uma classificação online protegida contra alterações. Ao evoluir para competição online, pontuação e autorização devem ser validadas no backend.

## Atores

Jogador cria ou escolhe um perfil, estuda e consulta desempenho. Autor de conteúdo mantém os arquivos JSON e revisa perguntas pelo repositório. Desenvolvedor mantém os módulos, valida dados e publica versões. Não existe painel administrativo conectado no MVP.

## Conteúdo mínimo

Sete fases de dez questões e uma final de quinze. Cada banco terá ao menos três vezes a quantidade sorteada por faixa: 15 fáceis, 9 médias e 6 difíceis em cada fase comum inicial; 9 fáceis, 18 médias e 18 difíceis na final. Questões podem ser compartilhadas com a final por tags, desde que todos os filtros e quotas continuem atendidos.

## 2 Arquitetura e organização

| Tecnologia            | Responsabilidade                                              |
|-----------------------|---------------------------------------------------------------|
| HTML5                 | Estrutura semântica, formulários, navegação e alternativas.   |
| CSS3                  | Layout, responsividade, contraste e animações.                |
| JavaScript ES modules | Estados, correção, sorteio, economia, objetivos e interface.  |
| JSON                  | Mundos, fases, perguntas, objetivos e configuração de regras. |
| SVG                   | Caminhos, bandeiras, portais e estados do mapa.               |
| localStorage          | Documento versionado de perfis, resultados e partida ativa.   |
| Git e GitHub          | Histórico e manutenção do código e conteúdo.                  |
| GitHub Pages          | Hospedagem dos arquivos estáticos.                            |

### Separação dos módulos

```

index.html
css/  style.css  home.css  map.css  quiz.css
js/
  app.js          navegação e composição das telas
  quiz.js         sessão respostas e feedback
  selection.js    sorteio e embaralhamento
  game.js         pontos XP moedas estrelas e níveis
  storage.js      salvar carregar migrar exportar importar
  achievements.js objetivos troféus e resgates
  map.js          fases pré requisitos e SVG
  ui.js           componentes e acesso por teclado
data/
  mundos.json    fases.json  perguntas.json
  objetivos.json  config.json
assets/ images/  icons/  sounds/
tests/ selection.test.js  game.test.js  storage.test.js
README.md

```

app.js carrega e valida o conteúdo, recupera o perfil e controla a navegação. selection.js devolve uma sessão com questões e alternativas ordenadas. quiz.js registra eventos. game.js calcula recompensas; achievements.js avalia objetivos; storage.js persiste o estado. A interface apresenta o resultado desses módulos, sem duplicar regras de negócio.

O jogo publicado não depende de um processo Node.js. Um servidor HTTP local pode ser usado no desenvolvimento para carregar módulos e JSON; abrir index.html diretamente pelo protocolo file não é o caminho de execução suportado.

### 3 Requisitos funcionais de navegação e jogo

#### **RF01 Iniciar jogo [M]**

Exibir Começar e Como jogar na tela inicial; Começar abre seleção ou criação de perfil.

#### **RF02 Identificar jogador [M]**

Criar perfil com apelido de 2 a 24 caracteres e avatar permitido; dispensar email e senha.

#### **RF03 Escolher mundo [M]**

Apresentar os cinco mundos e seu progresso; mostrar as regiões sem conteúdo como Em desenvolvimento.

#### **RF04 Exibir fases [M]**

Mostrar fases com número, objetivo, estado, recorde e estrelas no mapa e em lista equivalente.

#### **RF05 Exibir questão [M]**

Apresentar enunciado, código opcional e alternativas da sessão, com indicação de questão atual e total.

#### **RF06 Selecionar resposta [M]**

Permitir escolher uma alternativa e confirmar; exigir seleção antes da confirmação.

#### **RF07 Corrigir resposta [M]**

Comparar o ID selecionado com corretald e aceitar apenas uma resposta final por questão.

#### **RF08 Apresentar feedback [M]**

Após confirmação, mostrar acerto ou erro e os valores recebidos, sem avançar automaticamente.

#### **RF09 Mostrar explicação [M]**

Apresentar a resposta correta e explicação após acerto, erro ou pulo.

#### **RF10 Calcular pontuação [M]**

Aplicar pontos por dificuldade, bônus de sequência, penalidade de dica e recorde por fase.

#### **RF11 Calcular XP [M]**

Conceder XP por acerto e bônus elegíveis; não conceder XP por erro ou pulo.

#### **RF12 Controlar níveis [M]**

Derivar nível e barra de progresso do XP acumulado usando a tabela configurada.

#### **RF13 Controlar sequência [M]**

Incrementar sequência por acerto, zerar por erro ou pulo e guardar a maior sequência.

## 4 Requisitos funcionais de progresso e questões

### **RF14 Disponibilizar dicas [M]**

Mostrar dica após validar custo e limite; debitar uma vez e aplicar fator de pontos 0,7.

### **RF15 Registrar progresso [M]**

Guardar tentativas, aprovação, melhores resultados, estrelas e progresso dos mundos.

### **RF16 Salvar dados [M]**

Persistir estado versionado após cada ação confirmada e lidar com falha de gravação.

### **RF17 Exibir desempenho [M]**

Apresentar estatísticas por perfil e precisão calculada a partir dos resultados registrados.

### **RF18 Gerenciar conquistas [M]**

Avaliar condições, desbloquear troféus, mostrar progresso e permitir resgate único.

### **RF19 Reiniciar fase [M]**

Criar nova sessão com novo sorteio; manter recordes e histórico do perfil.

### **RF20 Continuar jogo [M]**

Recuperar perfil e partida ativa sem refazer o sorteio nem pagar novamente recompensas.

### **RF21 Sortear questões [M]**

Selecionar um subconjunto aleatório sem duplicatas e compatível com os filtros da fase.

### **RF22 Embaralhar alternativas [M]**

Alterar a ordem de alternativas embaralháveis mantendo seus IDs e a correção por identidade.

### **RF23 Controlar dificuldade [M]**

Atender exatamente às quotas de fácil, médio e difícil; bloquear fase com conteúdo insuficiente.

### **RF24 Evitar repetição recente [M]**

Priorizar perguntas fora das duas últimas tentativas e inéditas; usar as mais antigas quando necessário.

### **RF25 Registrar erros [M]**

Guardar tentativas e resultados por questão, incluindo exposições sem resposta.

### **RF26 Reforçar conteúdo [M]**

Priorizar erros pendentes dentro do conjunto não recente, sem quebrar quotas nem repetir na sessão.

## 5 Requisitos funcionais complementares do MVP

### **RF27 Gerenciar perfis locais [M]**

Criar, selecionar, renomear e excluir até dez perfis isolados no mesmo navegador.

### **RF28 Gerenciar CodeCoins [M]**

Manter saldo inicial de 100 moedas, créditos e débitos; impedir saldo negativo.

### **RF29 Atribuir estrelas [M]**

Aplicar 0, 1, 2 ou 3 estrelas conforme aprovação, precisão e uso de poderes.

### **RF30 Liberar fases e portais [M]**

Liberar próxima fase após aprovação e portal após final; manter bloqueio de conteúdo não publicado.

### **RF31 Pular questão [M]**

Cobrar 30 moedas uma vez por partida, registrar pulo com zero recompensa e mostrar explicação.

### **RF32 Exibir ranking local [M]**

Ordenar perfis por soma dos recordes de fases, estrelas e apelido; indicar alcance local.

### **RF33 Configurar experiência [M]**

Salvar preferências de som e movimento; oferecer controles acessíveis em todas as telas principais.

### **RF34 Exportar e importar progresso [M]**

Gerar JSON versionado, validar importação e obter decisão antes de substituir dados existentes.

### **RF35 Excluir progresso [M]**

Solicitar confirmação explícita ao apagar um perfil; preservar os demais perfis e dados de outros sites.

### **RF36 Tratar conteúdo inválido [M]**

Detectar IDs duplicados, resposta inexistente, filtros vazios ou quotas insuficientes antes de iniciar.

## 6 Requisitos funcionais de expansão

### **RF37 Temporizar desafios [E]**

Oferecer modo cronometrado e +Tempo, com regras separadas e acesso ao modo sem limite.

### **RF38 Controlar vidas [E]**

Iniciar modo específico com três vidas; encerrar após três erros e permitir nova tentativa.

### **RF39 Ampliar poderes [E]**

Implementar 50 por cento, segunda chance e troca limitada por questão equivalente.

### **RF40 Ampliar desafios [E]**

Adicionar verdadeiro ou falso, ordenar algoritmo e código livre com validação apropriada.

### **RF41 Renovar missões diárias [E]**

Criar objetivos por data e guardar resgates para evitar recompensa repetida.

### **RF42 Personalizar avatar [E]**

Permitir comprar ou desbloquear skins, títulos, molduras e temas com moedas.

### **RF43 Adaptar e revisar [E]**

Criar revisão dirigida e ajustar dificuldade por desempenho com regras transparentes.

### **RF44 Ampliar campanha [E]**

Publicar os demais mundos e permitir rotas opcionais com pré requisitos explícitos.

### **RF45 Conectar contas e ranking [E]**

Integrar backend para autenticação, sincronização, ranking online e administração.

### **RF46 Adicionar minijogos [E]**

Integrar movimento e desafios 2D com Phaser quando justificado pela experiência.

## 7 Regras de negócio

| Regra                   | Definição inicial                                                                                  |
|-------------------------|----------------------------------------------------------------------------------------------------|
| RN01 Aprovação          | Acertos divididos pelo total configurado da partida; aprovar a partir de 60%.                      |
| RN02 Pontos             | Fácil 100, médio 150, difícil 200; erro e pulo rendem zero.                                        |
| RN03 Combo              | Mais 10 por acerto anterior consecutivo, até 40; erro ou pulo zera.                                |
| RN04 Dica               | 20 moedas, uma por questão; pontos finais são 70% do base mais bônus, arredondados para baixo.     |
| RN05 XP e moedas        | Acerto fácil 50 XP e 10 moedas; médio 75 e 15; difícil 100 e 20.                                   |
| RN06 Primeira aprovação | Mais 100 XP e 50 moedas uma vez por fase e perfil.                                                 |
| RN07 Níveis             | Limites 0, 500, 1.200, 2.000 e 3.000; a partir do nível 5, mais 1.000 XP por nível.                |
| RN08 Estrelas           | 0 abaixo de 60%; 1 a partir de 60%; 2 a partir de 80%; 3 com 100% e nenhum poder usado.            |
| RN09 Recordes           | Preservar máximos de pontos, precisão e estrelas independentemente da última tentativa.            |
| RN10 Pulo               | 30 moedas, uma vez por partida; conta no denominador como não acertada e zera combo.               |
| RN11 Saldo              | Crédito inicial de 100 moedas por perfil; validar custo antes de debitar; nunca permitir negativo. |
| RN12 Ranking            | Soma dos melhores pontos de cada fase publicada; desempate por estrelas e apelido.                 |
| RN13 Objetivos          | Desbloquear quando condição for atingida; pagar recompensa uma vez ao resgatar.                    |
| RN14 Abandono           | Sem aprovação, estrelas ou bônus final; recompensas de respostas já confirmadas permanecem.        |
| RN15 Sequência          | Zerar no início da partida; persistir maior sequência da sessão e do perfil.                       |

A precisão de uma fase usa seu total previsto: seis acertos em dez questões, incluindo pulos, resultam em 60%. No perfil, precisão usa acertos divididos pelo total de resultados finalizados, incluindo pulos, e mostra 0% quando esse total é zero. Questões apenas exibidas e sem resposta não entram na precisão histórica.

Pontos de tentativas, XP e moedas são acumulados a cada confirmação válida; os bônus de primeira aprovação são pagos no encerramento elegível. A conclusão de uma sessão deve ser registrada uma única vez pelo ID de sessão, mesmo se a tela de resultado for recarregada.

## 8 Algoritmo de seleção dinâmica

Entradas: definição da fase, perguntas publicadas, histórico por perfil e sessões recentes. Saída: uma sessão persistente com IDs únicos, ordem de questões e ordem de alternativas. Não se sorteia novamente durante uma retomada.

### Procedimento

- Filtrar questões ativas por IDs da fase, tags ou temas permitidos e linguagem configurada.
- Separar por dificuldade e validar quantidade suficiente para as quotas. Se faltar conteúdo, mostrar erro de preparação; não alterar dificuldade silenciosamente.
- Dentro de cada quota, separar não recentes e recentes. Recente significa ter sido exibida em uma das duas últimas tentativas daquela fase.
- Selecionar primeiro questões não recentes. Enquanto houver inéditas e erros pendentes, alternar os grupos, começando pelas inéditas; se um grupo acabar, usar o outro.
- Completar com não recentes já acertadas. Em cada grupo, embaralhar com Fisher Yates e retirar sem reposição.
- Se não recentes forem insuficientes, usar recentes por exposição mais antiga; embaralhar empates. Manter dificuldade, tema e unicidade.
- Embaralhar a lista final. Embaralhar alternativas quando permitido e guardar as ordens resultantes.
- Persistir a sessão antes de exibir a primeira pergunta. Registrar exposição ao exibir cada questão, não apenas ao sorteá-la.

### Histórico e prioridades

Erro pendente é uma questão cuja última resposta finalizada foi incorreta. Depois de um acerto, ela deixa esse grupo. O pulo é registrado separadamente e não cria um erro de conteúdo. Uma tentativa abandonada participa do histórico recente somente com as questões efetivamente exibidas.

### Exemplo e limite da promessa

Num banco inicialmente sem histórico, com quotas que comportam três partidas, a janela recente permite três tentativas sem repetir perguntas. Depois, erros antigos podem retornar para revisão mesmo que ainda existam inéditas. O jogo não garante conjuntos totalmente distintos quando o histórico ou as quotas não permitirem. Sortear uma pergunta novamente não muda sua dificuldade.

### Embaralhamento seguro para correção

Cada alternativa possui ID estável; corretald nunca aponta para a posição visual A, B, C ou D. Alternativas como “todas as anteriores” devem ser evitadas ou ter embaralhamento desativado. Trechos idênticos ou variantes quase iguais precisam de revisão editorial; uma chave de família poderá limitar variantes numa expansão.

## 9 Modelo de conteúdo e configuração

| Entidade        | Campos essenciais                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------------|
| Mundo           | id, nome, ordem, descricao, faseId, publicado, visual.                                                                 |
| Fase            | id, mundold, ordem, titulo, objetivos, temas, linguagem, quantidade, quotas, prerequisites, final, publicada.          |
| Questão         | id, temas, dificuldade, tipo, linguagem, enunciado, codigo opcional, alternativas, corretald, explicacao, dica, ativa. |
| Alternativa     | id e texto; IDs únicos dentro da questão.                                                                              |
| Objetivo        | id, nome, descricao, tipoCondicao, meta, recompensaXP, recompensaMoedas, ativo.                                        |
| Configuração    | version, pontos, XP, moedas, níveis, custos, aprovação, estrelas, janelaRecente e limites.                             |
| Material visual | id, arquivo, autor, origem, licença ou permissão e finalidade.                                                         |

### Regras de validação

- IDs estáveis e únicos; nenhuma fase referencia mundo ou questão inexistente.
- Quotas inteiras e não negativas, cuja soma é quantidade.
- Cada questão do MVP possui quatro alternativas não vazias e exatamente um corretald válido.
- Explicação obrigatória; dica obrigatória quando o poder estiver habilitado para a questão.
- Dificuldade pertence a facil, medio ou dificil; tipo pertence ao catálogo implementado.
- Pré requisitos formam um grafo sem ciclos; fases futuras não entram na seleção jogável.
- Textos e código são dados; nomes não são usados como seletores executáveis ou HTML livre.

### Conteúdo editorial

O autor deve revisar linguagem, resposta, distratores e explicação. Perguntas de programação declaram a linguagem e o contexto necessário. Questões não devem depender de uma versão ou comportamento ambíguo sem explicitar essa condição. Revisões preservam IDs quando mantêm a mesma habilidade; mudanças substanciais podem exigir novos IDs.

A associação por temas permite reutilização controlada no desafio final. Os bancos continuam separados da lógica JavaScript. O esquema antigo com correta como índice deve ser convertido para alternativas com IDs antes de implementar o embaralhamento.

## 10 Exemplos de fase e questão em JSON

Os exemplos abaixo são válidos como registros individuais. Os arquivos finais armazenam coleções de registros e precisam conter questões suficientes para todas as quotas. A fase 04 segue a trilha de variáveis definida no GDD.

```
{
  "id": "camp60-04",
  "mundoId": "camp60",
  "ordem": 4,
  "titulo": "Variáveis e constantes",
  "objetivos": ["Reconhecer armazenamento de valores"],
  "temas": ["variaveis"],
  "linguagem": "javascript",
  "quantidade": 10,
  "quotas": {"facil": 5, "medio": 3, "dificil": 2},
  "prerequisitos": ["camp60-03"],
  "final": false,
  "publicada": true
}

{
  "id": "camp60-var-001",
  "temas": ["variaveis"],
  "dificuldade": "facil",
  "tipo": "multipla_escolha",
  "linguagem": "javascript",
  "enunciado": "O que é uma variável?",
  "alternativas": [
    {"id": "a", "texto": "Um nome associado a um valor no programa."},
    {"id": "b", "texto": "Um sistema operacional."},
    {"id": "c", "texto": "Uma linguagem de marcação."},
    {"id": "d", "texto": "Um dispositivo de entrada."}
  ],
  "corretaId": "a",
  "embaralharAlternativas": true,
  "explicacao": "Uma variável permite armazenar e utilizar valores.",
  "dica": "Pense em como o programa guarda uma informação.",
  "ativa": true
}
```

Se a opção correta for mostrada em C após o embaralhamento, seu ID continua sendo a. O clique registra o ID, e a correção usa corretald. Não se altera a pergunta de origem durante a partida.

## 11 Modelo de progresso e sessão

| Registro             | Conteúdo                                                                                                                         |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Raiz                 | schemaVersion, contentVersion, revision, activeProfileId, profiles.                                                              |
| Perfil               | id, apelido, avatarId, createdAt, xp, coins, totalPoints, stats, settings.                                                       |
| Fase por perfil      | approved, bestScore, bestAccuracy, bestStars, completedAttempts, firstBonusPaid.                                                 |
| Histórico de questão | shownCount, answeredCount, correctCount, wrongCount, skippedCount, lastShownAt, lastOutcome.                                     |
| Objetivos            | unlockedAt e claimedAt por ID; progresso derivado de estatísticas.                                                               |
| Sessão ativa         | sessionId, phaseId, contentVersion, questionIds, questionSnapshots, optionOrders, currentIndex, answers, streak, totals, status. |
| Histórico de sessões | ID, fase, início e fim, expostas, resultados e resumo, com retenção limitada.                                                    |

### Exemplo reduzido de sessão

```
{
  "sessionId": "sess-001",
  "phaseId": "camp60-04",
  "contentVersion": "1.0.0",
  "questionIds": ["camp60-var-001"],
  "optionOrders": {"camp60-var-001": ["d", "b", "a", "c"]},
  "currentIndex": 0,
  "answers": [],
  "streak": 0,
  "totals": {"points": 0, "xp": 0, "coins": 0},
  "usedPowers": [],
  "status": "awaiting_answer"
}
```

O exemplo apresenta uma questão para explicar o formato; a sessão real deve ter dez ou quinze conforme a fase. questionSnapshots guarda os dados necessários da partida para que uma atualização do banco não altere perguntas já sorteadas. A sessão é removida da área ativa depois de concluída ou abandonada.

### Invariantes

Uma resposta pertence a uma única posição e questão. Uma questão finalizada não aceita novo pagamento. Uma sessão encerrada não volta a ativa. Os totais do perfil e o resumo da sessão devem permanecer coerentes; ao recuperar estado inválido, o jogo informa o problema e oferece recuperação sem inventar resultados.

## 12 Persistência e estados da aplicação

### Salvar e recuperar

Usar uma chave própria, `codequizz:save:v1`, com um JSON contendo todos os perfis. A gravação substitui o documento inteiro depois de aplicar o evento em memória. Salvar após criação de perfil, uso de poder, confirmação de resposta, conclusão, resgate e alteração de configuração. Um evento confirmado não deve ser aplicado duas vezes.

`localStorage` conserva dados entre sessões na mesma origem, mas pode ser limpo pelo usuário, indisponível por política ou limitado por quota. O jogo deve avisar quando não consegue salvar e oferecer exportação. Não prometer continuidade em modo privado nem sincronização automática. Dados de HTTP e HTTPS pertencem a origens diferentes.

### Versões retenção e concorrência

`schemaVersion` controla migrações; `contentVersion` identifica regras e conteúdo. Antes de migrar, preservar cópia recuperável do `save`. Manter as vinte sessões encerradas mais recentes por perfil e estatísticas agregadas permanentes. Guardar a janela recente por fase separadamente para que a limpeza não altere as regras de sorteio.

Somente uma aba deve editar o perfil por vez. Usar `revision` e detectar mudanças pelo evento `storage`; se outra aba alterar o `save`, pausar ações e pedir recarga antes de continuar. Isso reduz conflitos, mas não substitui transações de backend. IDs removidos do banco permanecem no histórico até uma migração definida.

### Estados de uma questão e sessão

Preparação → aguardando resposta → feedback → próxima questão. Depois da última: resultado → concluída. Preparação pode falhar por dados inválidos. Aguardando resposta pode usar dica ou pulo; pulo leva ao feedback. Abandonar encerra a sessão sem bônus final. Uma sessão salva retoma exatamente no estado registrado.

### Exportação importação e exclusão

Exportar todo o `save` para JSON. Na importação, validar formato, tamanho máximo proposto de 2 MB, versões, IDs e valores numéricos; rejeitar arquivo incompatível sem apagar o atual. Mostrar resumo e pedir confirmação antes de substituir. Excluir perfil também exige confirmação e remove apenas seus registros. Não usar `localStorage.clear`, pois apagaria dados não pertencentes ao projeto.

## 13 Requisitos não funcionais

### RNF01 Compatibilidade

Validar versões estáveis atuais de Chrome, Edge e Firefox; incluir Safari quando houver ambiente disponível.

### RNF02 Responsividade

Operar de 360 px de largura até desktop; enunciado e controles sem rolagem horizontal, exceto código quando necessário.

### RNF03 Desempenho

Meta: interface inicial utilizável em até 3 s e início de fase em até 1 s após carregar dados, em rede de 15 Mbps e notebook de referência registrado no teste.

### RNF04 Usabilidade

Disponibilizar ação clara de continuar, voltar e iniciar; obter fluxo completo sem orientação direta em validação com iniciantes.

### RNF05 Legibilidade

Texto principal com tamanho mínimo proposto de 16 px na web; código com fonte monoespaçada e contraste adequado.

### RNF06 Persistência

Recuperar o último evento gravado após recarga e fechamento no mesmo navegador e origem.

### RNF07 Hospedagem

Funcionar como arquivos estáticos e com caminhos relativos ao repositório do GitHub Pages.

### RNF08 Manutenibilidade

Separar seleção, regras, armazenamento e interface; manter testes dos comportamentos críticos.

### RNF09 Organização

Manter conteúdo em JSON validado e assets com nomes estáveis e origem documentada.

### RNF10 Segurança

Não publicar credenciais; renderizar dados como texto; evitar eval e execução direta de código fornecido pelo jogador.

### RNF11 Acessibilidade

Usar HTML semântico, teclado, foco visível, nomes acessíveis e feedback textual; adotar contraste de 4,5 para 1 como meta do texto comum.

### RNF12 Movimento e áudio

Respeitar preferência por movimento reduzido e permitir jogar sem som.

### RNF13 Confiabilidade

Detectar save inválido, falha de JSON ou quota sem travar navegação; oferecer mensagem e recuperação.

### RNF14 Privacidade

Solicitar apenas apelido e preferências no MVP; permitir exportar e excluir os perfis locais.

### RNF15 Peso do conteúdo

Meta inicial de até 5 MB para primeira carga; comprimir imagens e carregar sons e regiões futuras sob demanda.

## 14 Cenários de aceitação do MVP

### CA01 Perfil e isolamento

Criar dois perfis, responder com um e trocar para outro. Cada perfil mantém saldo, XP e fases próprios. Rastreabilidade: RF02 RF27 RF28.

### CA02 Correção embaralhada

Exibir a resposta correta em outra posição e selecioná-la. Corrigir pelo ID e pagar uma única vez, inclusive após clique duplo. Rastreabilidade: RF06 RF07 RF22.

### CA03 Sorteio balanceado

Iniciar fase inicial com banco válido. Obter dez IDs únicos, nas quotas 5, 3 e 2 e apenas temas permitidos. Rastreabilidade: RF21 RF23.

### CA04 Repetição e erros

Simular histórico com recentes, inéditas e erros antigos. Respeitar janela, alternância e retorno controlado quando o banco se esgotar. Rastreabilidade: RF24 RF25 RF26.

### CA05 Quota insuficiente

Remover questões difíceis necessárias. Bloquear início com mensagem de conteúdo insuficiente sem reduzir a quota. Rastreabilidade: RF23 RF36.

### CA06 Dica e saldo

Usar dica em questão fácil com combo zero. Debitar 20, dar 70 pontos se acertar e impedir segundo débito. Rastreabilidade: RF10 RF14 RF28.

### CA07 Pulo

Pular uma questão: debitar 30, contar zero, zerar combo, mostrar explicação e impedir segundo pulo na sessão. Rastreabilidade: RF09 RF13 RF31.

### CA08 Estrelas e acesso

Testar 50%, 60%, 80% e 100% com e sem poder. Aplicar 0, 1, 2 e 3 conforme regras e liberar somente por aprovação. Rastreabilidade: RF29 RF30.

### CA09 Repetição de fase

Concluir e repetir com nota inferior. Preservar recorde; pagar bônus de primeira aprovação apenas uma vez. Rastreabilidade: RF15 RF19 RN06.

### CA10 Retomada e pagamentos

Recarregar durante feedback e na tela de resultado. Manter pergunta, ordem, pontos e resgates sem pagamento duplicado. Rastreabilidade: RF16 RF20.

### CA11 XP e níveis

Testar valores um ponto abaixo, exatamente e acima de cada limite, incluindo nível 6 com 4.000 XP. Rastreabilidade: RF11 RF12.

### CA12 Troféus e ranking

Atingir conquista, resgatar duas vezes e repetir fase. Pagar uma vez e classificar pela soma de melhores pontos. Rastreabilidade: RF18 RF32.

# 15 Qualidade publicação e documentação

## Aceitação de armazenamento e interface

- CA13 Importação: arquivo válido restaura dados após confirmação; arquivo inválido ou futuro é rejeitado sem apagar o save atual. RF34 e RNF13.
- CA14 Falha de salvamento: simular bloqueio e quota; comunicar perda de persistência e manter opção de exportar. RF16 e RNF13.
- CA15 Exclusão: cancelar preserva dados; confirmar remove só o perfil escolhido. RF35 e RNF14.
- CA16 Acesso: completar uma fase por teclado, em 360 px e sem áudio; ver foco, enunciado, explicação e resultado. RNF02, RNF11 e RNF12.
- CA17 Atualização: recuperar sessão com snapshot e testar migração do save sem alterar uma resposta concluída. RF20 e RNF06.
- CA18 Duas abas: editar o mesmo save em outra aba e verificar detecção de revisão antes de continuar. RNF13.

## Estratégia de teste

Testes unitários verificam seleção, identidade das alternativas, quotas, pontuação, níveis, estrelas e pagamentos únicos. Usar gerador aleatório controlável nos testes para reproduzir cenários. Testes de integração cobrem progresso, resgates, migração e retomada. Testes manuais cobrem fluxo completo, leitura, responsividade e teclado. Registrar ambiente, resultado e problemas; não afirmar aprovação de testes antes da execução.

## Publicação no GitHub Pages

Versionar os arquivos e escolher a fonte de publicação no repositório. Usar caminhos relativos para CSS, JavaScript, JSON e imagens, considerando a URL do projeto com nome do repositório. Validar a versão publicada por HTTPS, carregamento do JSON, links e continuidade do perfil. Caso use GitHub Actions, conferir permissões e regras do ambiente github-pages para a branch configurada.

O processo de build, quando houver, ocorre antes da publicação. GitHub Pages serve arquivos estáticos; o backend de uma versão futura precisa de hospedagem ou serviço próprio. O MVP não depende de npm run deploy: o procedimento final deve ser documentado conforme a fonte escolhida.

## README e histórico

Documentar objetivo, execução local por HTTP, estrutura, regras, escopo local dos dados, inclusão de perguntas e publicação. Commits sugeridos: feat: cria mapa do CAMP60; feat: adiciona sorteio balanceado; fix: preserva bônus de primeira aprovação. Manter versões de conteúdo e notas de migração.

# 16 Evolução riscos e referências

## Evolução prevista

A expansão de conteúdo completa FORT-57, PAGEM, LAB22 e DIMENSÃO. A expansão de mecânicas acrescenta missões diárias, ramificações, vidas, cronômetro, +Tempo, 50 por cento, segunda chance, troca de pergunta, avatares e novos desafios. A adaptação de dificuldade e a revisão dirigida usam o histórico por habilidade.

Uma versão conectada pode avaliar Firebase ou Supabase para contas, progresso e ranking. A escolha depende dos requisitos, custos e regras de acesso da época. A classificação online exige validação de resultados no servidor; acrescentar um banco sem validar ações não resolve a confiança nos pontos. Um painel de autoria e migração de perfis locais também precisarão de especificação.

Phaser pode ser avaliado para mapas com movimento, colisão e minijogos. Código livre exige um ambiente de execução isolado e limites próprios; não deve usar eval no contexto da página. Esses recursos não bloqueiam a entrega inicial.

## Riscos e tratamento

| Risco                     | Tratamento previsto                                                         |
|---------------------------|-----------------------------------------------------------------------------|
| Banco pequeno             | Produzir quotas suficientes e revisar cobertura antes de liberar fase.      |
| Memorização e ambiguidade | Sorteio, alternativas por ID, variantes por habilidade e revisão editorial. |
| Perda de dados locais     | Salvar por evento, exportar e importar; avisar sobre falhas.                |
| Recompensa repetida       | IDs de sessão, firstBonusPaid e claimedAt persistidos.                      |
| Sobrecarga visual         | Fundos moderados, contraste, teclado e testes em celular.                   |
| Escopo excessivo          | CAMP60 completo no MVP e demais recursos separados por versão.              |

## Referências técnicas consultadas

GitHub Docs. What is GitHub Pages. <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>. Acesso em 01 out. 2026.

MDN Web Docs. Window localStorage property. <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>. Acesso em 01 out. 2026.

Phaser. What is Phaser. <https://docs.phaser.io/phaser/getting-started/what-is-phaser>. Acesso em 01 out. 2026. Documento relacionado: CodeQuizz Game Design Document, versão documental 1.0.

## Anexo Infraestrutura do MVP

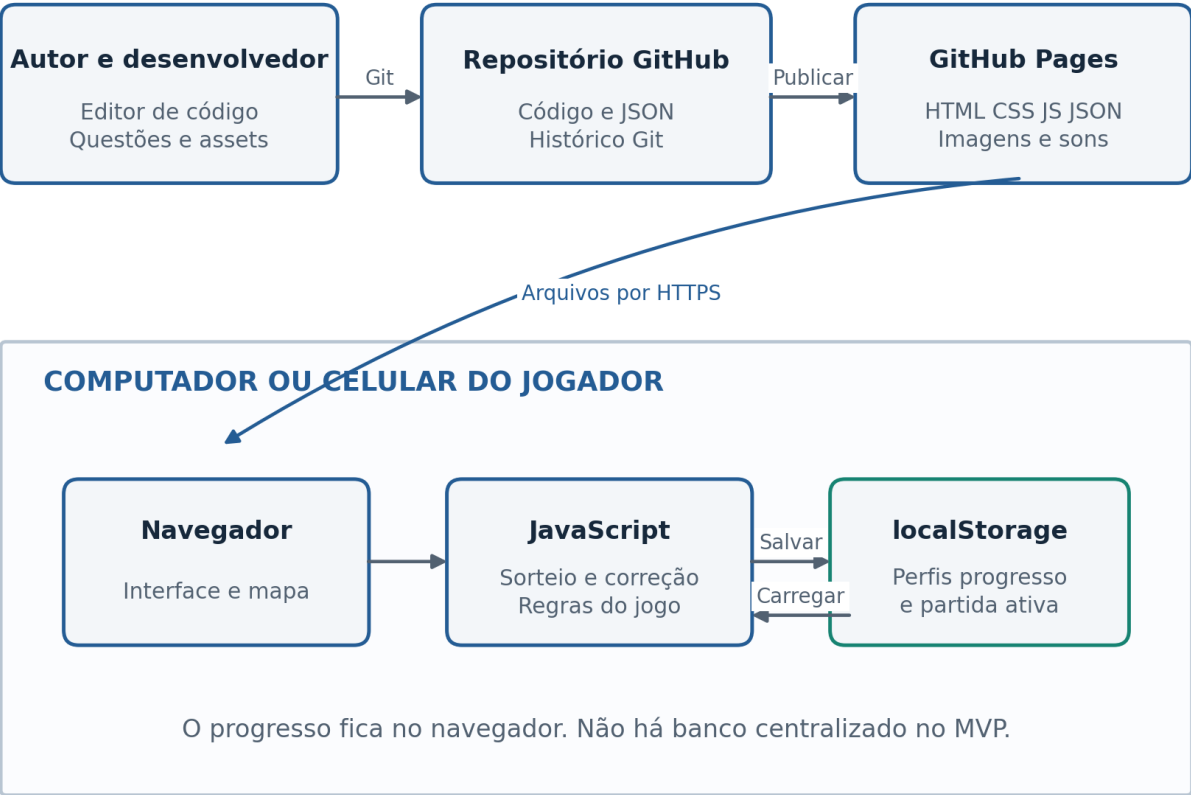


Figura 1 Infraestrutura definida para o MVP com distribuição de arquivos estáticos e progresso local.

O autor publica código e conteúdo no repositório. GitHub Pages disponibiliza esses arquivos pela internet. No computador ou celular, o navegador executa a lógica de sorteio, correção e recompensas. O save fica no localStorage dessa origem; não é enviado a um banco centralizado.

| Ambiente        | Componentes necessários                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------|
| Desenvolvimento | Editor, Git, navegador e servidor HTTP local para testes.                                     |
| Publicação      | Repositório GitHub e fonte de publicação do GitHub Pages.                                     |
| Jogador         | Navegador compatível, JavaScript e armazenamento local habilitados.                           |
| Sem conexão     | Não garantido no MVP; suporte offline completo exige cache e tratamento próprios em expansão. |

## Anexo Organização interna da aplicação

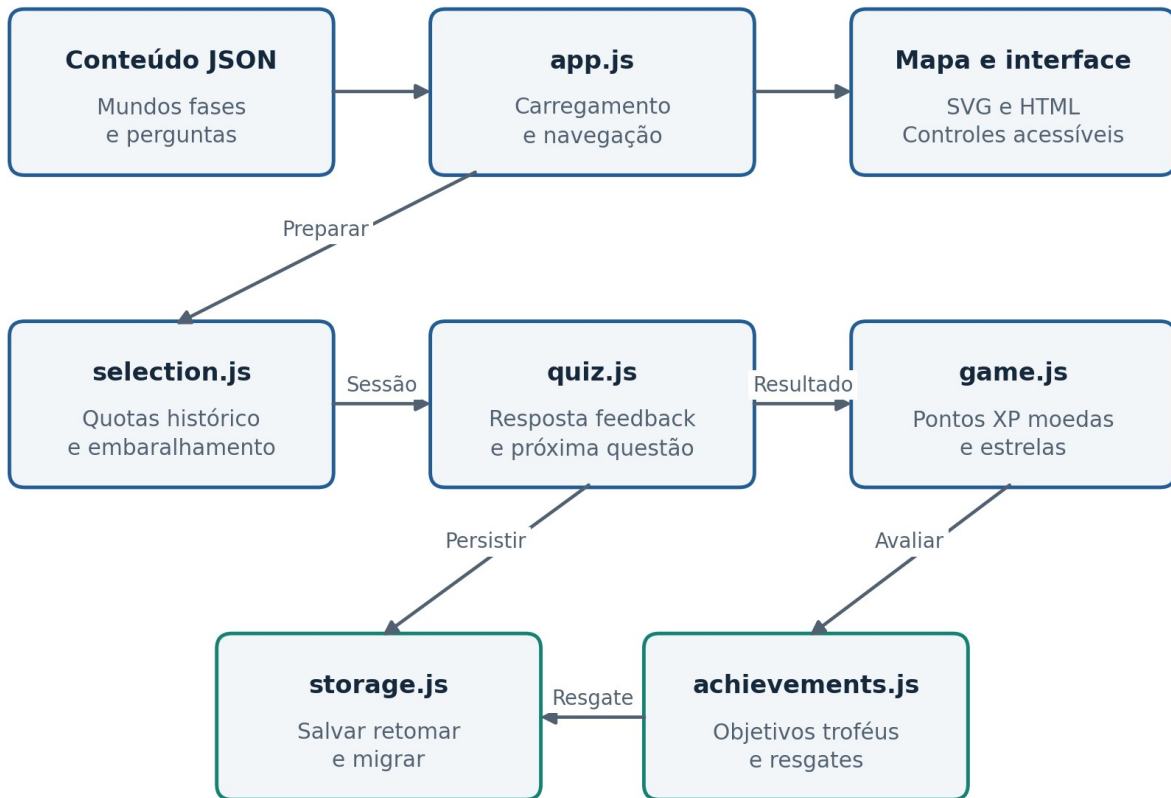


Figura 2 Responsabilidades principais dos módulos JavaScript e circulação de conteúdo e resultados.

O diagrama apresenta relações principais, sem listar todos os eventos de interface. **selection.js** recebe histórico do perfil para preparar a sessão. **game.js** calcula as recompensas, **achievements.js** avalia objetivos e **storage.js** grava o documento completo. A validação impede que um evento já aplicado seja creditado novamente.

## Anexo Infraestrutura da versão conectada

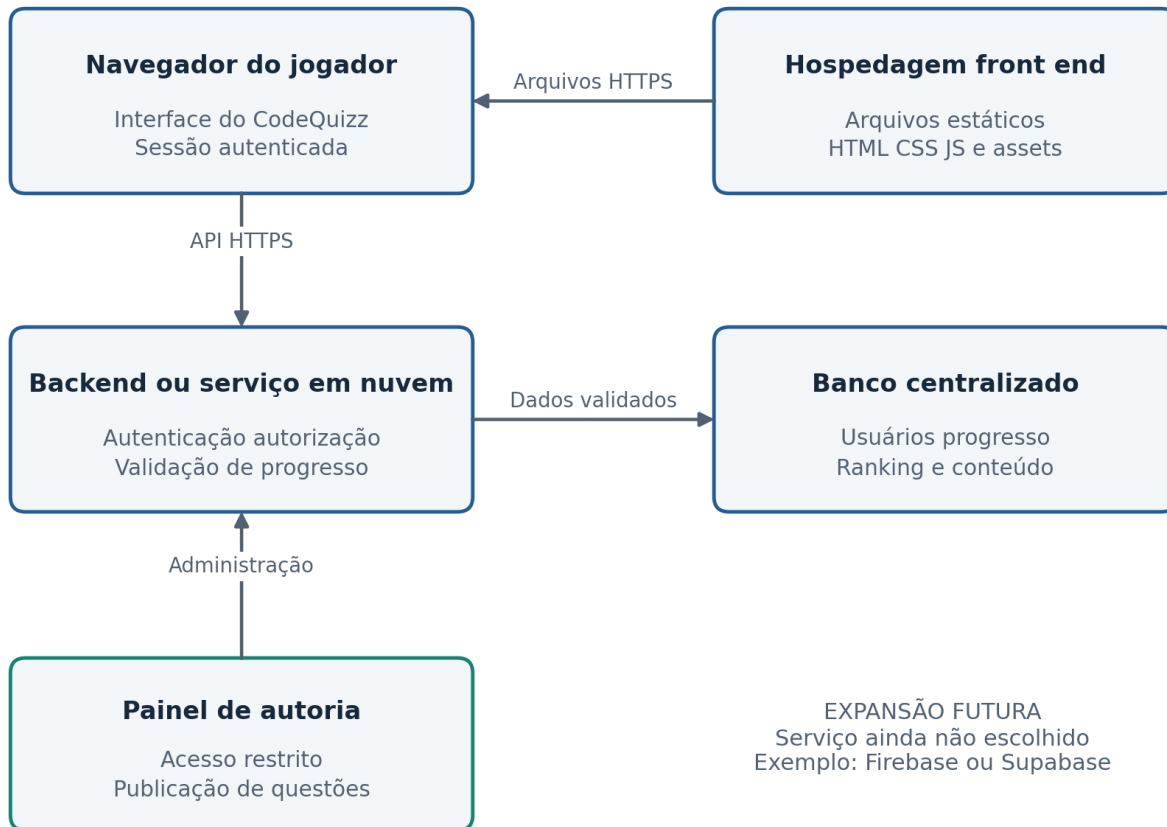


Figura 3 Possível arquitetura futura com backend autenticação e banco centralizado.

Esta imagem documenta uma expansão, sem serviço contratado ou infraestrutura já implantada. O front end pode continuar hospedado como arquivos estáticos. Um serviço separado autentica contas, controla permissões e valida progresso. O banco centraliza dados e viabiliza sincronização e ranking online. O painel de autoria exige acesso restrito.

Firebase e Supabase são opções para avaliação posterior. Antes de escolher, revisar custo, regras de segurança, migração dos perfis locais e modelo de validação de partidas. O objetivo é permitir uma experiência conectada sem comprometer a entrega inicial do jogo.